



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №5
по дисциплине «Программирование на языке Джава»

Выполнил студент группы **ИКБО-50-25**

Багинян А.В.

Принял
Старший преподаватель

Ермаков С.Р.

Москва 2026

1. Задачи к выполнению

1.1. Часть 1. Дженерики и обобщённое программирование

Дженерики (generics) позволяют обнаруживать ошибки уже во время компиляции программы, а не во время её выполнения, и позволяют параметризовать типы классов, интерфейсов и методов, начиная с JDK 1.5.

1.1.1. Задача 1

1. Напишите метод, которому передаётся коллекция объектов типа `ArrayList`, а возвращается коллекция `ArrayList`, но уже без дубликатов. Необходимо использовать метод `contains()` интерфейса `List`.
2. Реализуйте алгоритм линейного поиска элемента в массиве. При нахождении элемента необходимо вернуть его позицию в массиве. Если элемент не найден, то вернуть `-1`.
3. Реализуйте поиск наибольшего элемента в массиве с помощью метода `compareTo()` интерфейса `Comparable`. Определите класс `Circle` с полем `radius` и найдите наибольший элемент в массиве экземпляров этого класса.
4. Реализуйте поиск наибольшего элемента в двумерном массиве с помощью метода `compareTo()` интерфейса `Comparable`.

1.1.1.1. Решение

```
1 // generics/Circle.java
2 package ru.haykor.mirea.generics;
3
4 public class Circle implements Comparable<Circle> {
5     private double radius;
6
7     public Circle(double radius) {
8         this.radius = radius;
9     }
10
11     public double getRadius() {
12         return radius;
13     }
14
15     @Override
16     public int compareTo(Circle o) {
17         return Double.compare(radius, o.radius);
18     }
19 }
```

```

18     }
19
20     @Override
21     public String toString() {
22         return "Circle{radius=" + radius + "}";
23     }
24 }

```

```

1 // generics/CollectionUtils.java
2 package ru.haykor.mirea.generics;
3
4 import java.util.ArrayList;
5
6 public class CollectionUtils {
7
8     public static <E> ArrayList<E>
withoutDuplicates(ArrayList<E> list) {
9         ArrayList<E> result = new ArrayList<>();
10        for (E element : list) {
11            if (!result.contains(element)) {
12                result.add(element);
13            }
14        }
15        return result;
16    }
17
18    public static <E> int linearSearch(E[] array, E key) {
19        for (int i = 0; i < array.length; i++) {
20            if (array[i].equals(key)) {
21                return i;
22            }
23        }
24        return -1;
25    }
26
27    public static <E extends Comparable<E>> E findMax(E[]
array) {
28        E max = array[0];
29        for (int i = 1; i < array.length; i++) {
30            if (array[i].compareTo(max) > 0) {
31                max = array[i];
32            }
33        }

```

```

34         return max;
35     }
36
37     public static <E extends Comparable<E>> E findMax(E[][]
array) {
38         E max = array[0][0];
39         for (E[] row : array) {
40             for (E element : row) {
41                 if (element.compareTo(max) > 0) {
42                     max = element;
43                 }
44             }
45         }
46         return max;
47     }
48 }

```

```

1 // app/TestGenericAlgorithms.java
2 package ru.haykor.mirea.app;
3
4 import java.util.ArrayList;
5 import java.util.List;
6
7 import ru.haykor.mirea.generics.Circle;
8 import ru.haykor.mirea.generics.CollectionUtils;
9
10 public class TestGenericAlgorithms {
11     public static void main(String[] args) {
12         ArrayList<Integer> numbers = new
ArrayList<>(List.of(1, 2, 2, 3, 3, 3, 4));
13         System.out.println("Без дубликатов: " +
CollectionUtils.withoutDuplicates(numbers));
14
15         Integer[] array = {5, 3, 9, 1, 7};
16         System.out.println("Позиция 9: " +
CollectionUtils.linearSearch(array, 9));
17         System.out.println("Позиция 42: " +
CollectionUtils.linearSearch(array, 42));
18
19         Circle[] circles = {new Circle(2.0), new Circle(5.5),
new Circle(3.1)};
20         System.out.println("Наибольший круг: " +
CollectionUtils.findMax(circles));

```

```

21
22         Circle[][] circleGrid = {
23             {new Circle(1.0), new Circle(4.0)},
24             {new Circle(6.5), new Circle(2.0)}
25         };
26         System.out.println("Наибольший круг в матрице: " +
CollectionUtils.findMax(circleGrid));
27     }
28 }

```

1.2. Часть 2. Обобщённый стек

Стек — это структура данных, которая содержит данные по принципу «последним пришёл — первым обслужен» (LIFO). Для его моделирования удобно определить обобщённый класс `GenericStack<E>`, реализованный с помощью отношения композиции (поле-ссылка на `ArrayList<E>`):

```

1 // generics/GenericStack.java (базовая реализация через
композицию)
2 package ru.haykor.mirea.generics;
3
4 import java.util.ArrayList;
5
6 public class GenericStack<E> {
7     private ArrayList<E> list = new ArrayList<>();
8
9     public int getSize() {
10         return list.size();
11     }
12
13     public E peek() {
14         return list.get(getSize() - 1);
15     }
16
17     public void push(E o) {
18         list.add(o);
19     }
20
21     public E pop() {
22         E o = list.get(getSize() - 1);
23         list.remove(getSize() - 1);
24         return o;
25     }

```

```

26
27     public boolean isEmpty() {
28         return list.isEmpty();
29     }
30
31     @Override
32     public String toString() {
33         return "стек: " + list.toString();
34     }
35 }

```

1.2.1. Задача 2

1. Измените класс `GenericStack` таким образом, чтобы реализовать его с помощью массива, а не `ArrayList`. Перед добавлением нового элемента в стек необходимо проверить размер массива. Если массив заполнен, создайте новый массив, который удвоит текущий размер массива и скопирует элементы из текущего массива в новый.
2. Класс `GenericStack` из предыдущего описания реализован с помощью отношения композиции. Определите новый класс стека, который наследуется от `ArrayList`. Реализуйте новый класс `GenericStack`. Напишите тестовую программу, которая запросит у пользователя пять строк, а отобразит их в обратном порядке.

Таблица 1.1 — Отношения между вариантами обобщённого стека и `ArrayList<E>`

Класс	Отношение к <code>ArrayList<E></code>
<code>GenericStack<E></code>	композиция (has-a) — хранит <code>ArrayList<E></code> в приватном поле <code>list</code>
<code>GenericArrayStack<E></code>	не зависит от <code>ArrayList</code> — хранит элементы в собственном массиве <code>E[]</code>
<code>GenericStackByInheritance<E></code>	наследование (is-a) — расширяет <code>ArrayList<E></code> и наследует все его методы

Отношения между классами (см. Таблица 1.1): `GenericStack` моделирует стек через композицию, поэтому у него нет лишних унаследованных методов `ArrayList`; `GenericStackByInheritance`, наоборот, наследует все публичные методы `ArrayList` (в том числе не относящиеся к стеку, такие как

`add(int, E)` или `remove(Object)`), что демонстрирует недостаток моделирования стека через наследование.

1.2.1.1. Решение

```
1 // generics/GenericArrayStack.java
2 package ru.haykor.mirea.generics;
3
4 import java.util.Arrays;
5
6 public class GenericArrayStack<E> {
7     private E[] elements;
8     private int size;
9
10    @SuppressWarnings("unchecked")
11    public GenericArrayStack() {
12        elements = (E[]) new Object[10];
13    }
14
15    public int getSize() {
16        return size;
17    }
18
19    public E peek() {
20        return elements[size - 1];
21    }
22
23    public void push(E o) {
24        if (size == elements.length) {
25            elements = Arrays.copyOf(elements, elements.length
26 * 2);
27        }
28        elements[size++] = o;
29    }
30
31    public E pop() {
32        E o = elements[size - 1];
33        elements[--size] = null;
34        return o;
35    }
36
37    public boolean isEmpty() {
38        return size == 0;
39    }
39 }
```

```

39
40     @Override
41     public String toString() {
42         return "Стек: " +
Arrays.toString(Arrays.copyOf(elements, size));
43     }
44 }

```

```

1 // generics/GenericStackByInheritance.java
2 package ru.haykor.mirea.generics;
3
4 import java.util.ArrayList;
5
6 public class GenericStackByInheritance<E> extends ArrayList<E>
{
7
8     public E peek() {
9         return get(size() - 1);
10    }
11
12    public void push(E o) {
13        add(o);
14    }
15
16    public E pop() {
17        E o = get(size() - 1);
18        remove(size() - 1);
19        return o;
20    }
21 }

```

```

1 // app/TestGenericStackByInheritance.java
2 package ru.haykor.mirea.app;
3
4 import java.util.Scanner;
5
6 import ru.haykor.mirea.generics.GenericStackByInheritance;
7
8 public class TestGenericStackByInheritance {
9     public static void main(String[] args) {
10         Scanner scanner = new Scanner(System.in);
11         GenericStackByInheritance<String> stack = new

```

```

GenericStackByInheritance<>();
12
13     System.out.println("Введите пять строк:");
14     for (int i = 0; i < 5; i++) {
15         stack.push(scanner.nextLine());
16     }
17
18     System.out.println("В обратном порядке:");
19     while (!stack.isEmpty()) {
20         System.out.println(stack.pop());
21     }
22
23     scanner.close();
24 }
25 }

```

1.3. Часть 3. Клонирование объектов

Чтобы определить пользовательский класс, который реализует интерфейс `Cloneable`, этот класс должен переопределить метод `clone()`, определённый в классе `Object`. Метод `clone()` класса `Object` выполняет только поверхностную копию: если поле объектного типа, то копируется лишь ссылка на объект, а не его содержимое. Чтобы получить глубокую копию, поле-ссылку необходимо клонировать отдельно внутри переопределённого метода `clone()`.

1.3.1. Задача 1

1. Определите класс `MyStack` в соответствии со следующей таблицей:

Таблица 1.2 — Спецификация класса `MyStack`

Метод / поле	Описание
<code>-list: ArrayList</code>	Список для хранения элементов
<code>+isEmpty(): boolean</code>	Возвращает <code>true</code> , если стек пуст
<code>+getSize(): int</code>	Возвращает количество элементов в стеке
<code>+peek(): Object</code>	Возвращает элемент на вершине стека, не удаляя его
<code>+pop(): Object</code>	Возвращает и удаляет элемент на вершине стека
<code>+push(o: Object): void</code>	Добавляет элемент в верхнюю часть стека

Класс `MyStack` реализован с помощью композиции (поле `list` типа `ArrayList`). Убедитесь, что класс `MyStack` реализован с помощью композиции. Определите новый класс `MyStack`, который наследуется от класса `ArrayList`. Реализуйте класс `MyStack`. Напишите клиент класса `MyStack` — программу, которая запрашивает у пользователя пять строк и отображает их в обратном порядке.

2. Перепишите класс `MyStack` для выполнения глубокой копии поля списка.

Отношение между классами аналогично отношению между `GenericStack` и `ArrayList` из части 2: `MyStack` содержит `ArrayList` (композиция, has-a), а `MyStackByInheritance` наследуется от `ArrayList` (наследование, is-a). Оба класса используют «сырые» (raw) типы `ArrayList` и `Object` — без параметризации, как указано в задании.

1.3.1.1. Решение

```
1 // stack/MyStackByInheritance.java
2 package ru.haykor.mirea.stack;
3
4 import java.util.ArrayList;
5
6 public class MyStackByInheritance extends ArrayList {
7
8     public Object peek() {
9         return get(size() - 1);
10    }
11
12    public Object pop() {
13        Object o = get(size() - 1);
14        remove(size() - 1);
15        return o;
16    }
17
18    public void push(Object o) {
19        add(o);
20    }
21 }
```

```
1 // app/TestMyStack.java
2 package ru.haykor.mirea.app;
3
```

```

4 import java.util.Scanner;
5
6 import ru.haykor.mirea.stack.MyStackByInheritance;
7
8 public class TestMyStack {
9     public static void main(String[] args) {
10         Scanner scanner = new Scanner(System.in);
11         MyStackByInheritance stack = new
MyStackByInheritance();
12
13         System.out.println("Введите пять строк:");
14         for (int i = 0; i < 5; i++) {
15             stack.push(scanner.nextLine());
16         }
17
18         System.out.println("В обратном порядке:");
19         while (!stack.isEmpty()) {
20             System.out.println(stack.pop());
21         }
22
23         scanner.close();
24     }
25 }

```

Итоговый класс `MyStack` (композиция), уже дополненный методом `clone()` из задачи 2, выполняющим глубокую копию поля `list`:

```

1 // stack/MyStack.java
2 package ru.haykor.mirea.stack;
3
4 import java.util.ArrayList;
5
6 public class MyStack implements Cloneable {
7     private ArrayList list = new ArrayList();
8
9     public boolean isEmpty() {
10         return list.isEmpty();
11     }
12
13     public int getSize() {
14         return list.size();
15     }
16 }

```

```

17     public Object peek() {
18         return list.get(getSize() - 1);
19     }
20
21     public Object pop() {
22         Object o = list.get(getSize() - 1);
23         list.remove(getSize() - 1);
24         return o;
25     }
26
27     public void push(Object o) {
28         list.add(o);
29     }
30
31     @Override
32     public Object clone() {
33         try {
34             MyStack cloned = (MyStack) super.clone();
35             // Глубокая копия поля list, чтобы клон не делил
хранилище с оригиналом
36             cloned.list = (ArrayList) list.clone();
37             return cloned;
38         } catch (CloneNotSupportedException e) {
39             return null;
40         }
41     }
42
43     @Override
44     public String toString() {
45         return "стек: " + list.toString();
46     }
47 }

```

```

1 // app/TestMyStackClone.java
2 package ru.haykor.mirea.app;
3
4 import ru.haykor.mirea.stack.MyStack;
5
6 public class TestMyStackClone {
7     public static void main(String[] args) {
8         MyStack stack1 = new MyStack();
9         stack1.push("a");
10        stack1.push("b");

```

```

11     stack1.push("c");
12
13     MyStack stack2 = (MyStack) stack1.clone();
14     stack2.push("d");
15
16     System.out.println("stack1: " + stack1);
17     System.out.println("stack2: " + stack2);
18     System.out.println("Списки — разные объекты: " +
(stack1 != stack2));
19 }
20 }

```

Программа выводит `stack1: стек: [a, b, c]` и `stack2: стек: [a, b, c, d]` — после клонирования у `stack2` появляется соб-

ственный список, независимый от `list` объекта `stack1`, что подтверждает выполнение глубокой копии поля `list` (без переопределения `clone()` оба стека делили бы одну и ту же ссылку на `ArrayList`, и `push("d")` изменил бы оба объекта одновременно).